Entregable - Figure 6.19, ARM 3 y ARM 4

1) Figure 6.19

a) Archivo `Grammar.g4`

```
grammar Grammar;

program: stat+ ;

stat: ID '=' expr NEWLINE          # assignment
    | NEWLINE                      # blank
    ;

expr: expr op=('*'|'/') expr       # MulDiv
    | expr op=('+'|'-') expr       # AddSub
    | INT                          # int
    | ID                           # id
    | '(' expr ')'                 # parents
    ;

ID  : [a-zA-Z]+ ;
INT : [0-9]+ ;
NEWLINE:'\r'? '\n' ;
WS  : [ \t]+ -> skip ;
```

b) Archivo `Translator.php`

```php
<?php

class Attr {
    public ?string $AddrNUM = null;
    public ?string $AddrID = null;
    public ?string $Code = null;
}

class SymbolTable {
    private array $IDS = [];

    public function insert(string $name): void {
        $this->IDS[$name] = true;
    }

    public function exists(string $name): bool {
        return isset($this->IDS[$name]);
    }
}
```

```php
class Translator extends GrammarBaseVisitor {
    public int $TID = 0;
    public string $Data = ".global _start\n\n.data\n";
    public string $Code = "\n.text\n_start:\n";

    private SymbolTable $symbolTable;

    public function __construct() {
        $this->symbolTable = new SymbolTable();
    }

    public function isNUM(Attr $attr): bool {
        return $attr->AddrNUM !== null && $attr->AddrID === null;
    }

    public function visitProgram(Context\ProgramContext $context) {
        foreach ($context->stat() as $statContext) {
            $this->visit($statContext);
        }

        $this->Code .= "\tmov X8, #93\n\tsvc #0\n";
        $attr = new Attr();
        $attr->Code = $this->Data . $this->Code;
        return $attr;
    }

    public function visitAssignment(Context\AssignmentContext $context) {
        $id = $context->ID()->getText();
        $expr = $this->visit($context->expr());

        if ($this->isNUM($expr)) {
            if ($this->symbolTable->exists($id)) {
                $this->Code .= "\tldr X0, =" . $id . "\n";
                $this->Code .= "\tmov X1, #" . $expr->AddrNUM . "\n";
                $this->Code .= "\tstr X1, [X0]\n\n";
            } else {
                $this->Data .= $id . ": .word " . $expr->AddrNUM . "\n";
                $this->symbolTable->insert($id);
            }
        } else {
            if ($this->symbolTable->exists($id)) {
                $this->Code .= "\tldr X0, =" . $id . "\n";
                $this->Code .= "\tldr X1, =" . $expr->AddrID . "\n";
                $this->Code .= "\tldr X2, [X1]\n";
                $this->Code .= "\tstr X2, [X0]\n\n";
            } else {
                $this->Data .= $id . ": .word 0\n";
                $this->Code .= "\tldr X0, =" . $id . "\n";
                $this->Code .= "\tldr X1, =" . $expr->AddrID . "\n";
                $this->Code .= "\tldr X2, [X1]\n";
                $this->Code .= "\tstr X2, [X0]\n\n";
                $this->symbolTable->insert($id);
            }
        }
```

```
        return new Attr();
    }

    public function visitAddSub(Context\AddSubContext $context) {
        $left = $this->visit($context->expr(0));
        $right = $this->visit($context->expr(1));

        $this->TID++;
        $addr = "t" . $this->TID;
        $this->Data .= $addr . ": .word 0\n";

        $this->Code .= "\tldr X0, =" . $addr . "\n";

        $opText = $context->op->getText();
        $op = ($opText === "+") ? "add" : "sub";

        if ($this->isNUM($left)) {
            if ($this->isNUM($right)) {
                $this->Code .= "\tmov X1, #" . $left->AddrNUM . "\n";
                $this->Code .= "\t" . $op . " X1, X1, #" . $right->AddrNUM
. "\n";
                $this->Code .= "\tstr X1, [X0]\n";
            } else {
                $this->Code .= "\tmov X1, #" . $left->AddrNUM . "\n";
                $this->Code .= "\tldr X2, =" . $right->AddrID . "\n";
                $this->Code .= "\tldr X3, [X2]\n";
                $this->Code .= "\t" . $op . " X1, X1, X3\n";
                $this->Code .= "\tstr X1, [X0]\n";
            }
        } else {
            if ($this->isNUM($right)) {
                $this->Code .= "\tldr X1, =" . $left->AddrID . "\n";
                $this->Code .= "\tldr X1, [X1]\n";
                $this->Code .= "\t" . $op . " X1, X1, #" . $right->AddrNUM
. "\n";
                $this->Code .= "\tstr X1, [X0]\n";
            } else {
                $this->Code .= "\tldr X1, =" . $left->AddrID . "\n";
                $this->Code .= "\tldr X2, [X1]\n";
                $this->Code .= "\tldr X3, =" . $right->AddrID . "\n";
                $this->Code .= "\tldr X4, [X3]\n";
                $this->Code .= "\t" . $op . " X2, X2, X4\n";
                $this->Code .= "\tstr X2, [X0]\n";
            }
        }

        $this->Code .= "\n";
        $attr = new Attr();
        $attr->AddrID = $addr;
        return $attr;
    }

    public function visitMulDiv(Context\MulDivContext $context) {
```

```
        $left = $this->visit($context->expr(0));
        $right = $this->visit($context->expr(1));

        $this->TID++;
        $addr = "t" . $this->TID;
        $this->Data .= $addr . ": .word 0\n";

        $this->Code .= "\tldr X0, =" . $addr . "\n";

        $opText = $context->op->getText();
        $op = ($opText === "*") ? "mul" : "sdiv";

        if ($this->isNUM($left)) {
            if ($this->isNUM($right)) {
                $this->Code .= "\tmov X1, #" . $left->AddrNUM . "\n";
                $this->Code .= "\tmov X2, #" . $right->AddrNUM . "\n";
                $this->Code .= "\t" . $op . " X1, X1, X2\n";
                $this->Code .= "\tstr X1, [X0]\n";
            } else {
                $this->Code .= "\tmov X1, #" . $left->AddrNUM . "\n";
                $this->Code .= "\tldr X2, =" . $right->AddrID . "\n";
                $this->Code .= "\tldr X3, [X2]\n";
                $this->Code .= "\t" . $op . " X1, X1, X3\n";
                $this->Code .= "\tstr X1, [X0]\n";
            }
        } else {
            if ($this->isNUM($right)) {
                $this->Code .= "\tldr X1, =" . $left->AddrID . "\n";
                $this->Code .= "\tldr X2, [X1]\n";
                $this->Code .= "\tmov X3, #" . $right->AddrNUM . "\n";
                $this->Code .= "\t" . $op . " X2, X2, X3\n";
                $this->Code .= "\tstr X2, [X0]\n";
            } else {
                $this->Code .= "\tldr X1, =" . $left->AddrID . "\n";
                $this->Code .= "\tldr X2, [X1]\n";
                $this->Code .= "\tldr X3, =" . $right->AddrID . "\n";
                $this->Code .= "\tldr X4, [X3]\n";
                $this->Code .= "\t" . $op . " X2, X2, X4\n";
                $this->Code .= "\tstr X2, [X0]\n";
            }
        }

        $this->Code .= "\n";
        $attr = new Attr();
        $attr->AddrID = $addr;
        return $attr;
    }

    public function visitParents(Context\ParentsContext $context) {
        return $this->visit($context->expr());
    }

    public function visitId(Context\IdContext $context) {
        $id = $context->ID()->getText();
```

```
        $attr = new Attr();
        $attr->AddrID = $id;
        return $attr;
    }

    public function visitInt(Context\IntContext $context) {
        $num = $context->INT()->getText();
        $attr = new Attr();
        $attr->AddrNUM = $num;
        return $attr;
    }
}
```

c) Archivo `index.php`

```
<?php

require __DIR__ . '/../vendor/autoload.php';

require_once __DIR__ . '/GrammarLexer.php';
require_once __DIR__ . '/GrammarParser.php';
require_once __DIR__ . '/GrammarVisitor.php';
require_once __DIR__ . '/GrammarBaseVisitor.php';
require_once __DIR__ . '/Translator.php';

use Antlr\Antlr4\Runtime\InputStream;
use Antlr\Antlr4\Runtime\CommonTokenStream;

$input = $argv[1] ?? null;

if (!$input) {
    echo "Usage: php index.php \"code\"\n";
    exit(1);
}

$stream = InputStream::fromString($input);
$lexer = new GrammarLexer($stream);
$tokens = new CommonTokenStream($lexer);
$parser = new GrammarParser($tokens);

$tree = $parser->program();

$visitor = new Translator();
$result = $visitor->visit($tree);

echo "ARM64 code:\n" . $result->Code . "\n";
```

d) Archivo `index.sh`

```
#!/bin/sh

cd "$(dirname "$0")"

#php index.php "a = 1  b = 2 * a"

#php index.php "a = 1  b = 2  c = a + b * 2"

php index.php "a = 1  b = 2  c = a + b * 2  d = (c + 1) * 3"
```

e) Salida generada (ARM64) al ejecutar `index.sh`

f) Imagenes de resultados ARM

f.1) Resultado ARM - parte 1

```
isai@IsaiPatzan-deskop:~/Documentos/Github/dragon_examples_antlr4_php$ /bin/sh "/home/isai/Documentos/Github/dragon_e
xamples_antlr4_php/figure_6.19/index.sh"
ARM64 code:
.global _start

.data
a: .word 1
t1: .word 0
b: .word 0

.text
_start:
        ldr X0, =t1
        mov X1, #2
        ldr X2, =a
        ldr X3, [X2]
        mul X1, X1, X3
        str X1, [X0]

        ldr X0, =b
        ldr X1, =t1
        ldr X2, [X1]
        str X2, [X0]

        mov X8, #93
        svc #0
```

f.2) Resultado ARM - parte 2

```
● isai@IsaiPatzan-deskop:~/Documentos/Github/dragon_examples_antlr4_php$ /bin
  xamples_antlr4_php/figure_6.19/index.sh"
  ARM64 code:
  .global _start

  .data
  a: .word 1
  b: .word 2
  t1: .word 0
  t2: .word 0
  c: .word 0

  .text
  _start:
          ldr X0, =t1
          ldr X1, =b
          ldr X2, [X1]
          mov X3, #2
          mul X2, X2, X3
          str X2, [X0]

          ldr X0, =t2
          ldr X1, =a
          ldr X2, [X1]
          ldr X3, =t1
          ldr X4, [X3]
          add X2, X2, X4
          str X2, [X0]

          ldr X0, =c
          ldr X1, =t2
          ldr X2, [X1]
          str X2, [X0]

          mov X8, #93
          svc #0
```

f.3) Resultado ARM - parte 3

```
isai@IsaiPatzan-deskop:~/Documentos/Github/dragon_examples_antlr4_php$ /bin/sh
xamples_antlr4_php/figure_6.19/index.sh"
ARM64 code:
.global _start

.data
a: .word 1
b: .word 2
t1: .word 0
t2: .word 0
c: .word 0
t3: .word 0
t4: .word 0
d: .word 0

.text
_start:
        ldr X0, =t1
        ldr X1, =b
        ldr X2, [X1]
        mov X3, #2
        mul X2, X2, X3
        str X2, [X0]

        ldr X0, =t2
        ldr X1, =a
        ldr X2, [X1]
        ldr X3, =t1
        ldr X4, [X3]
        add X2, X2, X4
        str X2, [X0]

        ldr X0, =c
        ldr X1, =t2
        ldr X2, [X1]
        str X2, [X0]

        ldr X0, =t3
        ldr X1, =c
```

```
        ldr X1, =c
        ldr X1, [X1]
        add X1, X1, #1
        str X1, [X0]

        ldr X0, =t4
        ldr X1, =t3
        ldr X2, [X1]
        mov X3, #3
        mul X2, X2, X3
        str X2, [X0]

        ldr X0, =d
```

```
        ldr X1, =t4
        ldr X2, [X1]
        str X2, [X0]

        mov X8, #93
        svc #0
```

2) ARM 3

a) Configuracion de la Tarea 3

```
 GNU nano 7.2                    test.s
.global _start

_start:
        mov x0, #5

        mov x8, #93
        svc #0
```

```
pi@raspberrypi:~/arm64$ gcc test.s -o test
/usr/bin/ld: /tmp/ccF6Q9od.o: in function `_start':
(.text+0x0): multiple definition of `_start'; /usr/lib/gcc/aarch64-linux-gnu/12/../../../aarch64-linux-gnu/Scrt1.o:(.tex
t+0x0): first defined here
/usr/bin/ld: /usr/lib/gcc/aarch64-linux-gnu/12/../../../aarch64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1c): undefined reference to `main'
/usr/bin/ld: (.text+0x20): undefined reference to `main'
collect2: error: ld returned 1 exit status
pi@raspberrypi:~/arm64$
```

```
pi@raspberrypi:~/arm64$ objdump -s test

test:     file format elf64-littleaarch64

Contents of section .text:
 400078 a00080d2 a80b80d2 010000d4           ............
pi@raspberrypi:~/arm64$
```

b) Ejercicio 1

```
pi@raspberrypi:~/arm64$ ls
asld  test  test.o  test.s
pi@raspberrypi:~/arm64$ bash asld test
5
```

c) Ejercicio 2

```
pi@raspberrypi:~/arm64$ ls
02_equ.s  asld  test.s
pi@raspberrypi:~/arm64$ bash asld 02_equ
100
pi@raspberrypi:~/arm64$
```

d) Ejercicio 3

```
pi@raspberrypi:~/arm64$ ls
01_test.s  02_equ.s  03_align.s  asld
pi@raspberrypi:~/arm64$ bash asld 03_align
0
pi@raspberrypi:~/arm64$
```

e) Ejercicio 4

```
pi@raspberrypi:~/arm64$ bash asldec 04_align16
pi@raspberrypi:~/arm64$ objdump -s 04_align16

04_align16:     file format elf64-littleaarch64

Contents of section .text:
 4000b0 a80b80d2 010000d4 1f2003d5 1f2003d5  ......... ... ..
Contents of section .data:
 4100c0 01000000 02000000 00000000 00000000  ................
pi@raspberrypi:~/arm64$
```

f) Ejercicio 5

```
pi@raspberrypi:~/arm64$ bash asldec 05_ldr
pi@raspberrypi:~/arm64$ ls
01_test.s  03_align.s  04_align16.s  05_ldr.s  asldec
02_equ.s   04_align16  05_ldr        asld
pi@raspberrypi:~/arm64$
```

3) ARM 4

a) Ejercicio 1

```
.global _start

.data
a: .word 0xFFFFFFFF00000000
b: .word 0xFFFFFFFF

.text
_start:
    ldr x1, =a        // x1 = addr of a
    ldr x2, =b        // x2 = addr of b
    ldr x3, [x2]      // x3 = content of b
    str x3, [x1]      // a = b OR a (assuming the intention was a = b)
    ldrb w0, [x1]     // return a (loads the lowest byte of 'a' into w0)

    mov x8, #93       // exit syscall number (93 is the number for `exit` in Linux AArch64)
    svc #0            // execute syscall
```

b) Ejercicio 2

```
.global _start

_start:
    mov x0, #1        // set stdout (File Descriptor 1 for stdout)
    ldr x1, =msg      // load msg (Pointer to the "Hello, World!\n" string)
    mov x2, #14       // set msg size (14 bytes, including the newline \n)
    mov x8, #64       // write code (Linux syscall number 64 for `write`)
    svc #0            // syscall (Execute the `write` syscall)

    mov x0, #0        // return value (Exit code 0, success)

    mov x8, #93       // exit (Linux syscall number 93 for `exit`)
    svc #0            // syscall (Execute the `exit` syscall)

.data
msg: .asciz "Hello, World!\n"
```

```
isai@IsaiPatzan-deskop:~/Documentos/Github/ARQUI1_LAB/Tarea4$ bash compile.sh build exec
✅ Ensamblado completado.
🔗 Enlazando objetos...
✅ Enlazado completado: main
🎯 Ejecutable listo.
🚀 Ejecutando programa...
Hello, World!
```

c) Ejercicio 3

```
_start:
    mov x0, #0       // set stdin (File Descriptor 0)
    ldr x1, =buf     // load buffer (Pointer to the 'buf' storage area)
    mov x2, #16      // size buffer (Maximum 16 bytes to read)
    mov x8, #63      // read syscall_num (Linux syscall #63)
    svc #0           // generic syscall

    // **3. Imprimir el contenido del buffer (Echo)**
    mov x0, #1       // set stdout (File Descriptor 1)
    ldr x1, =buf     // load buffer (Pointer to the read data)
    // NOTE: The size (x2) is typically the value returned by the 'read' syscall,
    // which overwrites x0, but here it is omitted, assuming the original 'read' size, or
    // it expects the programmer to manually set x2 again. Image 1 shows x2 is missing.
    // In practice, x0 (return value of read) should be moved to x2.
    // Assuming the program intends to print the full buffer of 16 bytes.
    mov x8, #64      // write syscall_num (Linux syscall #64)
    svc #0           // generic syscall

    // **4. Salir del programa (Exit)**
    mov x0, #0       // return value (Exit code 0, success)
    mov x8, #93      // exit syscall_num (Linux syscall #93)
    svc #0           // generic syscall

.data
msg: .ascii "Enter a text: " // The prompt string

.bss
buf: .space 16               // 16 bytes of uninitialized space for the bufferbuf
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN PUERTOS GITLENS TERMINAL

```
isai@IsaiPatzan-deskop:~/Documentos/Github/ARQUI1_LAB/Tarea4$ bash compile.sh build exec
🚀 Ejecutando programa...
Enter a text: hola
hola
```

d) Ejercicio 4

```
.global _start

.data
ptr: .word 0            // pointer variable (stores the address returned by brk)

.text
_start:
    // **1. Obtener la dirección actual del break (brk(NULL))**
    mov x8, #214          // syscall brk (Linux syscall #214)
    mov x0, #0           // brk(NULL) - passing 0 returns the current break address
    svc #0               // generic syscall. The address is returned in x0.

    // **2. Guardar la dirección del break en la variable 'ptr'**
    ldr x1, =ptr         // x1 = &ptr (load the address of the pointer variable)
    str x0, [x1]         // ptr = x0 (store the brk address into the 'ptr' variable)

    // **3. Reservar 4 bytes de memoria dinámica**
    add x0, x0, #4       // x0 = x0 + 4 (Calculate the new break address: current + 4 bytes)
    mov x8, #214          // syscall brk (Linux syscall #214)
    svc #0               // generic syscall (Set the new break limit, reserving 4 bytes)

    // **4. Escribir el valor 100 en la memoria asignada (*ptr = 100)**
    ldr x1, =ptr         // x1 = &ptr (load the address of the 'ptr' variable)
    ldr x2, [x1]         // x2 = *x1 = ptr (load the value of 'ptr', which is the address of the reserved memory)
    mov x3, #100          // x3 = 100 (load the value 100)
    str x3, [x2]         // *ptr = 100 (store 100 at the address held in x2)

    // **5. Demostrar la lectura de la memoria asignada**
    ldr x1, =ptr         // x1 = &ptr (load the address of the 'ptr' variable)
```

```
    ldr x2, [x1]          // x2 = *x1 = ptr (load the address of the reserved memory)
    ldr x0, [x2]          // x0 = *ptr (load the content of the reserved memory, which is 100)

    // **6. Salir del programa (exit)**
    mov x8, #93           // syscall exit
    svc #0                // generic syscall. The exit code is the value in x0 (which is 100)
```

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   PUERTOS   GITLENS   TERMINAL        bash - Tarea4

isai@IsaiPatzan-deskop:~/Documentos/Github/ARQUI1_LAB/Tarea4$ bash compile.sh build exec
🔗 Enlazando objetos...
✅ Enlazado completado: main
🎯 Ejecutable listo.
🚀 Ejecutando programa...
```

e) Ejercicio 5

```
.global _start

.data
a: .word -1, -2, -3

.text
_start:
    ldr x0, =a        // x0 = &a (Carga la dirección base del array 'a')
    ldr x1, [x0]      // x1 = a[0] (Carga el primer elemento: -1)
    ldr x2, [x0, #4]  // x2 = a[1] (Carga el segundo elemento: -2, usando un desplazamiento de 4 bytes)
    ldr x3, [x0, #8]  // x3 = a[2] (Carga el tercer elemento: -3, usando un desplazamiento de 8 bytes)

    add x0, x1, x2    // x0 = x1 + x2. Resultado: -1 + -2 = -3
    add x0, x0, x3    // x0 = x0 + x3. Resultado: -3 + -3 = -6
    neg x0, x0        // x0 = -x0. Resultado: -(-6) = 6

    mov x8, #93       //exit
    svc #0

    // (Instrucciones de salida omitidas, pero el resultado final es 6 en x0)
```